



软件系统设计复习

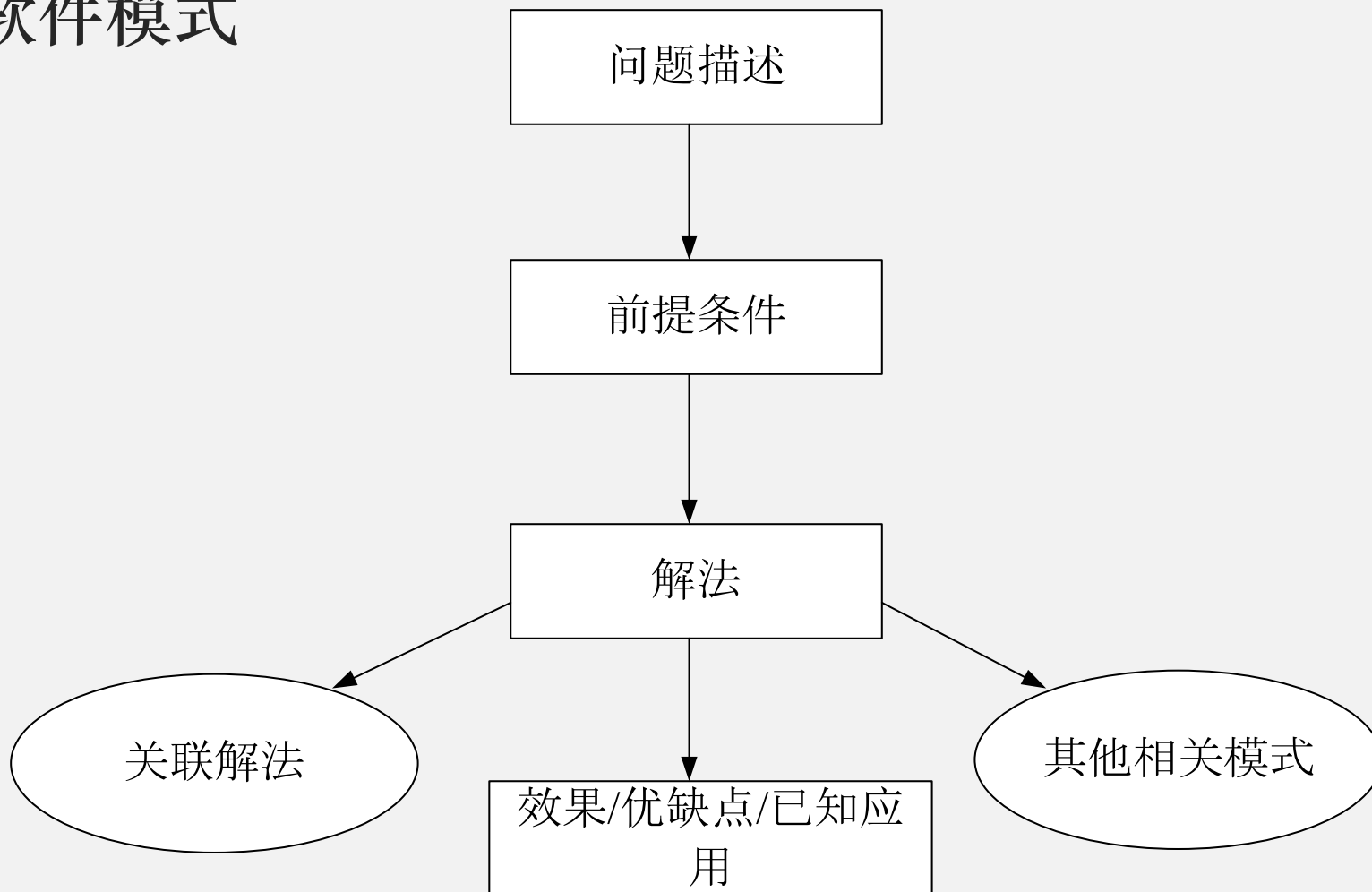
软件模式

- 软件模式

- 软件模式是将模式的一般概念应用于软件开发领域，即**软件开发的总体指导思路或参照样板**。软件模式并非仅限于设计模式，还包括架构模式、分析模式和过程模式等，实际上，**在软件生存期的每一个阶段都存在着一些被认同的模式**。
- 软件模式可以认为是对软件开发这一特定“问题”的“解法”的某种统一表示，**软件模式等于一定条件下的出现的问题以及解法**。软件模式的基础结构由4个部分构成：**问题描述、前提条件（环境或约束条件）、解法和效果**。

软件模式

- 软件模式



软件模式

- 软件模式
 - 软件模式与具体的应用领域无关，在模式发现过程中需要遵循大三律(Rule of Three)，即只有经过三个以上不同类型（或不同领域）的系统的校验，一个解决方案才能从候选模式升格为模式。

设计模式的定义

- 设计模式的定义
 - 设计模式(Design Pattern)是一套被反复使用、多数人知晓的、经过分类编目的、代码设计经验的总结，使用设计模式是为了可重用代码、让代码更容易被他人理解、保证代码可靠性。

设计模式的基本要素

- 设计模式的基本要素
 - 设计模式一般有如下几个基本要素：模式名称、问题、目的、解决方案、效果、实例代码和相关设计模式，其中的关键元素包括以下四个方面：
 - 模式名称 (Pattern name)
 - 问题 (Problem)
 - 解决方案 (Solution)
 - 效果 (Consequences)

设计模式的分类

- 设计模式的分类

- 根据其目的（模式是用来做什么的）可分为创建型(Creational)，结构型(Structural)和行为型(Behavioral)三种：
 - 创建型模式主要用于创建对象。
 - 结构型模式主要用于处理类或对象的组合。
 - 行为型模式主要用于描述对类或对象怎样交互和怎样分配职责。

设计模式的分类

• 设计模式的分类

- 根据**范围**，即模式主要是用于处理类之间关系还是处理对象之间的关系，可分为**类模式**和**对象模式**两种：
 - 类模式**处理类和子类之间的关系**，这些关系通过继承建立，在编译时刻就被确定下来，是属于**静态**的。
 - 对象模式**处理对象间的关系**，这些关系在运行时刻变化，更具**动态**性。

设计模式的分类

范围\目的	创建型模式	结构型模式	行为型模式
类模式	工厂方法模式	（类）适配器模式	模板方法模式
对象模式	抽象工厂模式 建造者模式 原型模式 单例模式	（对象）适配器模式 桥接模式 组合模式 装饰模式 外观模式 享元模式 代理模式	命令模式 迭代器模式 中介者模式 观察者模式 状态模式 策略模式

设计模式与类库框架

Q: If design patterns are so great, why can't someone build a library of them so I don't have to?

A: Design patterns are higher level than libraries. Design patterns tell us how to structure classes and objects to solve certain problems and it is our job to adapt those designs to fit our particular application.

Q: Aren't libraries and frameworks also design patterns?

A: Frameworks and libraries are not design patterns; they provide specific implementations that we link into our code. Sometimes, however, libraries and frameworks make use of design patterns in their implementations. That's great, because once you understand design patterns, you'll more quickly understand APIs that are structured around design patterns.

设计原则

- 对于面向对象的软件系统设计来说，在支持可维护性的同时，需要提高系统的可复用性。
- 软件的复用可以提高软件的开发效率，提高软件质量，节约开发成本，恰当的复用还可以改善系统的可维护性。
- 单一职责原则要求在软件系统中，一个类只负责一个功能领域中的相应职责。
- 开闭原则要求一个软件实体应当对扩展开放，对修改关闭，即在不修改源代码的基础上扩展一个系统的行为。
- 里氏代换原则可以通俗表述为在软件中如果能够使用基类对象，那么一定能够使用其子类对象。

设计原则

- 依赖倒转原则要求抽象不应该依赖于细节，细节应该依赖于抽象；要针对接口编程，不要针对实现编程。
- 接口隔离原则要求客户端不应该依赖那些它不需要的接口，即将一些大的接口细化成一些小的接口供客户端使用。
- 合成复用原则要求复用时尽量使用对象组合，而不使用继承。
- 迪米特法则要求一个软件实体应当尽可能少的与其他实体发生相互作用。

设计原则

- 目标：开闭原则
- 指导：最小知识原则
- 基础：单一职责原则、可变性封装原则
- 实现：依赖倒转原则、合成复用原则、里氏代换原则、接口隔离原则

表驱动法

- 目标
- 原理
- 如何快速从表中查询条目
 - 直接访问
 - 索引访问
 - 阶梯访问